

Lux Workspace Model: A Z Specification

Formal Model of Frames, Client Identity, and World Menu

March 2026

Contents

1	Introduction	2
2	Basic Types	2
3	Free Types	2
4	Global Constants	2
5	Workspace State	3
6	Initialization	3
7	Operations	4
7.1	Accept Connection	4
7.2	Identify Client	4
7.3	Create Frame	5
7.4	Add Scene to Frame	5
7.5	Update Scene in Frame	6
7.6	Close Frame (User)	6
7.7	Minimize Frame	7
7.8	Restore Frame	7
7.9	Register Menu Item	8
7.10	Deregister Menu Item	8
7.11	Disconnect Client	8
8	System Invariants Summary	9

1 Introduction

DES-022 introduces a workspace model for the Lux display server, inspired by Pharo/Squeak’s live environment. The display becomes a *workspace* containing independent, movable *frames* (ImGui inner windows). Each frame is owned by a connected client and contains one or more *scenes*.

This specification models the workspace layer that sits above the existing scene renderer (specified in `display-server.tex`). It covers:

- Client identity from the connection handshake
- Frame lifecycle: creation, minimization, restoration, closure
- Frame ownership and cleanup on client disconnect
- World menu: per-client namespaced menu registration
- Scene-to-frame targeting

We abstract away the element tree inside scenes (already modeled) and focus on the new state that DES-022 introduces.

2 Basic Types

$[FD, FRAMEID, SCENEID, CLIENTNAME, MENUID, MENUPATH]$

FD models socket file descriptors (shared with the display-server spec). $FRAMEID$ is the identifier a client assigns to a frame. $SCENEID$ identifies a scene within a frame. $CLIENTNAME$ is the human-readable display name declared during the connection handshake (e.g. “Lux”, “Vox”, “Quarry”). $MENUID$ and $MENUPATH$ identify menu items and their nesting path in the World menu.

3 Free Types

$ZBOOL ::= ztrue \mid zfalse$

Frame visibility state:

$FrameVis ::= fvNormal \mid fvMinimized$

Menu target — which menu surface an item belongs to:

$MenuTarget ::= mtTools \mid mtWorld$

4 Global Constants

Animation-friendly bounds. Production values would be larger; these are sized for ProB’s finite state space.

$maxClients : \mathbb{N}$
$maxFrames : \mathbb{N}$
$maxScenes : \mathbb{N}$
$maxMenuItems : \mathbb{N}$
$maxClients = 3$
$maxFrames = 4$
$maxScenes = 6$
$maxMenuItems = 6$

5 Workspace State

The workspace tracks connected clients with their identities, frames with their ownership and visibility, scene-to-frame mappings, and per-client menu registrations. All fields are flattened for ProB animatability.

Workspace

```

clients :  $\mathbb{P} FD$ 
clientNames :  $FD \rightarrow CLIENTNAME$ 
frames :  $\mathbb{P} FRAMEID$ 
frameOwner :  $FRAMEID \rightarrow FD$ 
frameVis :  $FRAMEID \rightarrow FrameVis$ 
frameScenes :  $FRAMEID \rightarrow \mathbb{P} SCENEID$ 
sceneFrame :  $SCENEID \rightarrow FRAMEID$ 
menuItems :  $\mathbb{P} MENUID$ 
menuOwner :  $MENUID \rightarrow FD$ 
menuTarget :  $MENUID \rightarrow MenuTarget$ 
menuPath :  $MENUID \rightarrow MENUPATH$ 

```

```

dom clientNames  $\subseteq$  clients
#clients  $\leq$  maxClients
dom frameOwner = frames
dom frameVis = frames
dom frameScenes = frames
ran frameOwner  $\subseteq$  clients
#frames  $\leq$  maxFrames
dom sceneFrame =  $\bigcup(\text{ran } \textit{frameScenes})$ 
 $\forall f : \textit{frames} \bullet \forall s : \textit{frameScenes } f \bullet \textit{sceneFrame } s = f$ 
#(dom sceneFrame)  $\leq$  maxScenes
dom menuOwner = menuItems
dom menuTarget = menuItems
dom menuPath = menuItems
ran menuOwner  $\subseteq$  clients
#menuItems  $\leq$  maxMenuItems

```

The key invariants are:

1. **Named clients are connected:** Only connected clients can have display names.
2. **Frame owners are connected:** Every frame is owned by a connected client. Disconnecting a client removes its frames.
3. **Frame metadata coverage:** Every frame has an owner, visibility state, and scene set.
4. **Scene-frame consistency:** Every scene knows which frame it belongs to, and that mapping is consistent with the frame's scene set.
5. **Menu owners are connected:** Every menu item is owned by a connected client.

6 Initialization

Init

Workspace'

$clients' = \emptyset$
 $clientNames' = \emptyset$
 $frames' = \emptyset$
 $frameOwner' = \emptyset$
 $frameVis' = \emptyset$
 $frameScenes' = \emptyset$
 $sceneFrame' = \emptyset$
 $menuItems' = \emptyset$
 $menuOwner' = \emptyset$
 $menuTarget' = \emptyset$
 $menuPath' = \emptyset$

7 Operations

7.1 Accept Connection

A new client connects. It has no display name yet (the **ConnectMessage** arrives later).

AcceptConnection

$\Delta Workspace$

$newClient? : FD$

$newClient? \notin clients$
 $\#clients < maxClients$
 $clients' = clients \cup \{newClient?\}$
 $clientNames' = clientNames$
 $frames' = frames$
 $frameOwner' = frameOwner$
 $frameVis' = frameVis$
 $frameScenes' = frameScenes$
 $sceneFrame' = sceneFrame$
 $menuItems' = menuItems$
 $menuOwner' = menuOwner$
 $menuTarget' = menuTarget$
 $menuPath' = menuPath$

7.2 Identify Client

A connected client declares its display name via **ConnectMessage**. Subsequent calls update the name (idempotent).

IdentifyClient

$\Delta \text{Workspace}$

$fd? : FD$

$name? : CLIENTNAME$

$fd? \in clients$

$clientNames' = clientNames \oplus \{fd? \mapsto name?\}$

$clients' = clients$

$frames' = frames$

$frameOwner' = frameOwner$

$frameVis' = frameVis$

$frameScenes' = frameScenes$

$sceneFrame' = sceneFrame$

$menuItems' = menuItems$

$menuOwner' = menuOwner$

$menuTarget' = menuTarget$

$menuPath' = menuPath$

7.3 Create Frame

A client sends a **SceneMessage** with a **frame_id** that does not yet exist. The display creates a new frame containing the specified scene.

CreateFrame

$\Delta \text{Workspace}$

$fd? : FD$

$frameId? : FRAMEID$

$sceneId? : SCENEID$

$fd? \in clients$

$frameId? \notin frames$

$sceneId? \notin \text{dom } sceneFrame$

$\#frames < maxFrames$

$\#(\text{dom } sceneFrame) < maxScenes$

$frames' = frames \cup \{frameId?\}$

$frameOwner' = frameOwner \cup \{frameId? \mapsto fd?\}$

$frameVis' = frameVis \cup \{frameId? \mapsto fvNormal\}$

$frameScenes' = frameScenes \cup \{frameId? \mapsto \{sceneId?\}\}$

$sceneFrame' = sceneFrame \cup \{sceneId? \mapsto frameId?\}$

$clients' = clients$

$clientNames' = clientNames$

$menuItems' = menuItems$

$menuOwner' = menuOwner$

$menuTarget' = menuTarget$

$menuPath' = menuPath$

7.4 Add Scene to Frame

A client sends a **SceneMessage** targeting an existing frame with a new scene. The scene is added (creating a tab if this is the second scene in the frame).

AddSceneToFrame

$\Delta \text{Workspace}$

$fd? : FD$

$frameId? : FRAMEID$

$sceneId? : SCENEID$

$fd? \in clients$

$frameId? \in frames$

$frameOwner\ frameId? = fd?$

$sceneId? \notin \text{dom } sceneFrame$

$\#(\text{dom } sceneFrame) < maxScenes$

$frameScenes' = frameScenes \oplus \{frameId? \mapsto (frameScenes\ frameId? \cup \{sceneId?\})\}$

$sceneFrame' = sceneFrame \cup \{sceneId? \mapsto frameId?\}$

$frames' = frames$

$frameOwner' = frameOwner$

$frameVis' = frameVis$

$clients' = clients$

$clientNames' = clientNames$

$menuItems' = menuItems$

$menuOwner' = menuOwner$

$menuTarget' = menuTarget$

$menuPath' = menuPath$

7.5 Update Scene in Frame

A client sends a **SceneMessage** targeting an existing frame with an existing scene ID. The scene content is replaced (modeled here as a no-op on workspace state since scene *content* is in the display-server spec).

UpdateScene

$\Xi \text{Workspace}$

$fd? : FD$

$frameId? : FRAMEID$

$sceneId? : SCENEID$

$fd? \in clients$

$frameId? \in frames$

$frameOwner\ frameId? = fd?$

$sceneId? \in frameScenes\ frameId?$

7.6 Close Frame (User)

The user clicks the close button on a frame. The frame and all its scenes are removed. An interaction event is sent to the owning client (modeled at the display-server level).

CloseFrame

$\Delta \text{Workspace}$

frameId? : *FRAMEID*

frameId? $\in$ *frames*
frames' = *frames* $\setminus$ {*frameId?*}
frameOwner' = {*frameId?*} $\triangleleft$ *frameOwner*
frameVis' = {*frameId?*} $\triangleleft$ *frameVis*
frameScenes' = {*frameId?*} $\triangleleft$ *frameScenes*
sceneFrame' = (*frameScenes* *frameId?*) $\triangleleft$ *sceneFrame*
clients' = *clients*
clientNames' = *clientNames*
menuItems' = *menuItems*
menuOwner' = *menuOwner*
menuTarget' = *menuTarget*
menuPath' = *menuPath*

7.7 Minimize Frame

MinimizeFrame

$\Delta \text{Workspace}$

frameId? : *FRAMEID*

frameId? $\in$ *frames*
frameVis *frameId?* = *fvNormal*
frameVis' = *frameVis* $\oplus$ {*frameId?* $\mapsto$ *fvMinimized*}
frames' = *frames*
frameOwner' = *frameOwner*
frameScenes' = *frameScenes*
sceneFrame' = *sceneFrame*
clients' = *clients*
clientNames' = *clientNames*
menuItems' = *menuItems*
menuOwner' = *menuOwner*
menuTarget' = *menuTarget*
menuPath' = *menuPath*

7.8 Restore Frame

RestoreFrame

$\Delta \text{Workspace}$

frameId? : *FRAMEID*

frameId? $\in$ *frames*
frameVis *frameId?* = *fvMinimized*
frameVis' = *frameVis* $\oplus$ {*frameId?* $\mapsto$ *fvNormal*}
frames' = *frames*
frameOwner' = *frameOwner*
frameScenes' = *frameScenes*
sceneFrame' = *sceneFrame*
clients' = *clients*
clientNames' = *clientNames*
menuItems' = *menuItems*
menuOwner' = *menuOwner*
menuTarget' = *menuTarget*
menuPath' = *menuPath*

7.9 Register Menu Item

A client registers a menu item. The `menuTarget` determines whether it appears in the Tools menu or the World menu (per-client namespace from handshake identity).

RegisterMenuItem $\Delta \text{Workspace}$ $fd? : FD$ $itemId? : MENUID$ $target? : MenuTarget$ $path? : MENUPATH$
$fd? \in clients$ $itemId? \notin menuItems$ $\#menuItems < maxMenuItems$ $menuItems' = menuItems \cup \{itemId?\}$ $menuOwner' = menuOwner \cup \{itemId? \mapsto fd?\}$ $menuTarget' = menuTarget \cup \{itemId? \mapsto target?\}$ $menuPath' = menuPath \cup \{itemId? \mapsto path?\}$ $clients' = clients$ $clientNames' = clientNames$ $frames' = frames$ $frameOwner' = frameOwner$ $frameVis' = frameVis$ $frameScenes' = frameScenes$ $sceneFrame' = sceneFrame$

7.10 Deregister Menu Item

A client removes a menu item it owns.

$\text{DeregisterMenuItem}$ $\Delta \text{Workspace}$ $fd? : FD$ $itemId? : MENUID$
$fd? \in clients$ $itemId? \in menuItems$ $menuOwner\ itemId? = fd?$ $menuItems' = menuItems \setminus \{itemId?\}$ $menuOwner' = \{itemId?\} \triangleleft menuOwner$ $menuTarget' = \{itemId?\} \triangleleft menuTarget$ $menuPath' = \{itemId?\} \triangleleft menuPath$ $clients' = clients$ $clientNames' = clientNames$ $frames' = frames$ $frameOwner' = frameOwner$ $frameVis' = frameVis$ $frameScenes' = frameScenes$ $sceneFrame' = sceneFrame$

7.11 Disconnect Client

A client disconnects. All its frames, scenes, and menu items are removed. This is the critical cleanup operation that maintains the ownership invariants.

Let *deadFrames* be the set of frames owned by the departing client, and *deadScenes* be all scenes in those frames, and *deadMenus* be all menu items owned by the client.

DisconnectClient

$\Delta \text{Workspace}$

deadClient? : *FD*

$deadClient? \in clients$

$clients' = clients \setminus \{deadClient?\}$

$clientNames' = \{deadClient?\} \triangleleft clientNames$

$frames' = frames \setminus \{f : frames \mid frameOwner\ f = deadClient?\}$

$frameOwner' = frames' \triangleleft frameOwner$

$frameVis' = frames' \triangleleft frameVis$

$frameScenes' = frames' \triangleleft frameScenes$

$sceneFrame' = sceneFrame \triangleright frames'$

$menuItems' = menuItems \setminus \{m : menuItems \mid menuOwner\ m = deadClient?\}$

$menuOwner' = menuItems' \triangleleft menuOwner$

$menuTarget' = menuItems' \triangleleft menuTarget$

$menuPath' = menuItems' \triangleleft menuPath$

8 System Invariants Summary

The key properties maintained across all operations:

1. **Frame owners are connected:** $\text{ran } frameOwner \subseteq clients$. No orphaned frames.
2. **Scene-frame consistency:** Every scene maps to exactly one frame, and that frame lists the scene in its scene set. No orphaned scenes.
3. **Menu owners are connected:** $\text{ran } menuOwner \subseteq clients$. No orphaned menu items.
4. **Disconnect atomicity:** When a client disconnects, all its frames, scenes, and menu items are removed in one atomic operation.
5. **Bounded resources:** Client count, frame count, scene count, and menu item count are all bounded.
6. **Named clients are connected:** $\text{dom } clientNames \subseteq clients$. Names cannot outlive connections.
7. **Ownership exclusivity:** Each frame has exactly one owner ($frameOwner$ is a total function on $frames$). Only the owning client can add scenes to a frame.